Design System Plan

1) User Interface (UI) Design

Goal: Simple, clear, web-based interface for enrolling and logging in with **face** and **gesture**.

Screens:

- **Welcome / Choose Login Type:** Select Gesture or Face; link to Register.
- **Register (Choose Methods):** Tick boxes for Gesture and Face (allow both).
- **Gesture Enrolment:** Webcam preview with landmarks overlay, progress/stability indicators, capture 2–3 samples, confirmation screen.
- **Face Enrolment:** Webcam preview with on-screen prompts (blink/turn), quality tips (lighting, framing), confirmation screen.
- **Login (Face):** Face detection with liveness prompt; shows pass/fail; link to switch to Gesture.
- **Login (Gesture):** Perform saved sequence; clear feedback; retry guidance.
- **Result / Lockout:** Success message, or failure count and cooldown timer.
- **Settings:** Re-enrol or delete templates; view which methods are active.

Style & Accessibility: Centered layout, 640×480 camera view, Material-style buttons, high contrast, concise status messages.

2) Backend Design

Pattern: Web REST API (stateless). Layers: Controller → Service → Repository. Sessions via JWT.

Responsibilities:

- **Registration & Enrolment:** Receive processed features (face/gesture), validate quality, store secure templates.
- **Login Verification:** Accept modality scores/proofs, compare against stored templates, apply policy (OR/AND/Weighted), return result.
- **Policies:** Central thresholds, fusion mode, lockout and cooldown settings.
- **Auditing:** Record attempts (time, outcome, reason) for security review.
- **Privacy & Security:** Store only transformed/hashed templates, validate liveness proofs, rate-limit requests.

Data Flow (web): Browser captures → computes scores → sends to API → API verifies → DB stores templates/logs → API responds with allow/deny.

3) Database Design

DBMS: MySQL (phpMyAdmin via XAMPP). **Design is extensible** so new password types can be added later without changing tables.

Key Entities & Roles:

- **users:** basic user records.
- **auth_profiles:** one active profile per user; ties a user to their chosen methods.
- **modalities:** lookup table of supported methods (FACE, GESTURE, etc.).
- **auth_templates:** one row per enrolled method (stores secure template and metadata).
- **login_sessions:** tracks current authentication sessions/challenges.
- **attempts:** logs each authentication attempt and outcome.
- **policies:** stores thresholds, fusion mode, lockout/cooldown.

ER Overview (text): users 1-n auth_profiles; auth_profiles 1-n auth_templates (→ modalities); auth_profiles 1-n login_sessions; login_sessions 1-n attempts; policies linked to profiles or global.

Benefits: Single schema works for face **and** gesture today, and supports adding more methods later by inserting a new modality entry—no schema change required.